

WBEM, WS-Management e Revisão Geral

Infraestruturas de Computação na Nuvem, Aula 13

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Agenda (1/2)

- Limitações do modelo SNMP e motivação para alternativas mais ricas.
- WBEM e o Common Information Model (CIM).
- O esquema CIM: classes, associações e herança.
- Operações WBEM e transporte.
- WS-Management: gestão baseada em standards web (SOAP).

Agenda (2/2)

- Comparação SNMP vs WBEM vs WS-Management.
- Gestão fora-de-banda: IPMI e Redfish.
- A tendência atual: APIs REST e modelos declarativos.
- Revisão geral do semestre, bloco a bloco (C1-C6).
- Preparação para o teste escrito e para o miniprojeto.

Para Além do SNMP

- Retomando a Aula 12: o SNMP baseia-se numa MIB simples, uma árvore plana de OIDs sem relações explícitas entre objetos.
- Adequado para contadores e estados simples, mas limitado para modelar sistemas complexos (ex. relação entre um servidor, os seus discos, sistemas de ficheiros e serviços).
- Nos anos 1990/2000 surgem standards orientados a objetos, capazes de representar essas relações: WBEM/CIM.
- Mais tarde, a gestão baseada em web services dá origem ao WS-Management.

WBEM: Arquitetura

- **WBEM** (Web-Based Enterprise Management): iniciativa da DMTF para unificar a gestão de sistemas heterogéneos.
- **CIMOM** (CIM Object Manager): servidor central que recebe pedidos de clientes e comunica com **providers**.
- **Providers**: módulos que traduzem entre o modelo CIM e o recurso real (disco, processo, serviço de rede).
- **Clientes**: aplicações de gestão que consultam o CIMOM (ex. `wbemcli`, `pywbem`).

CIM: Common Information Model

- O CIM define um esquema orientado a objetos: **classes, propriedades, métodos e associações**.
- Ao contrário da MIB do SNMP (árvore plana), o CIM modela explicitamente relações – ex. CIM_LogicalDisk associado a CIM_ComputerSystem.
- Permite navegar do geral (sistema) para o específico (disco, processo, serviço), sem conhecer OIDs de memória.
- Transporte histórico: **CIM-XML sobre HTTP**.

O Esquema CIM: Classes e Herança

- O CIM organiza-se em três camadas: modelo **core** (conceitos base), modelos **comuns** (sistemas, redes, dispositivos) e extensões **de fabricante**.
- Suporta **herança**: CIM_LogicalDisk deriva de classes mais gerais, herdando propriedades comuns a qualquer dispositivo de armazenamento.
- Cada instância é identificada por uma **chave** composta pelas propriedades marcadas como Key.
- As classes têm **métodos**, não apenas propriedades: é possível *invocar ações* (ex. desmontar um volume), e não só ler e escrever valores.
- Esta é uma diferença estrutural face ao SNMP, onde só existem operações genéricas de leitura e escrita sobre variáveis.

Associações: o que Distingue o CIM

- Uma **associação** é ela própria uma classe, que relaciona duas outras classes.
- Exemplo: uma associação liga um CIM_ComputerSystem aos CIM_LogicalDisk que lhe pertencem.
- Isto permite *navegar* o modelo: partindo de um sistema, descobrir os seus discos; partindo de um disco, descobrir a que sistema pertence.
- Na MIB do SNMP, uma relação destas teria de ser inferida por convenção de índices – não está explicitamente representada.
- É esta capacidade de modelar relações que justifica a maior complexidade do CIM.

MIB Plana vs Modelo CIM

SNMP — árvore plana de OIDs

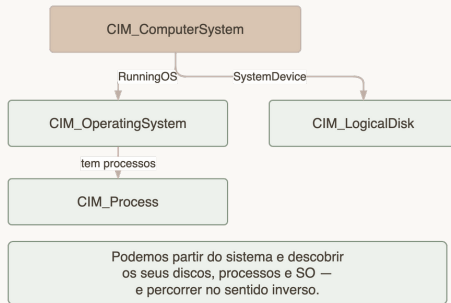
1.3.6.1.2.1.1.5.0
sysName

1.3.6.1.2.1.25.2.3.1
hrStorage

1.3.6.1.2.1.2.2.1.2
ifDescr

Valores isolados. A relação entre eles tem de ser inferida por convenção de índices — não está representada.

CIM — objetos com associações explícitas



Operações WBEM e Transporte

- **EnumerateClassNames** / **EnumerateInstances**: descobrir que classes existem e que instâncias delas estão presentes.
- **GetInstance** e **ModifyInstance**: ler e alterar uma instância concreta.
- **Associators** e **References**: percorrer as associações a partir de um objeto.
- **InvokeMethod**: executar um método definido numa classe.
- Transporte clássico: **CIM-XML** sobre HTTP(S), tipicamente no porto 5989 quando cifrado.
- **WBEM Indications**: mecanismo de notificação assíncrona, equivalente conceptual às traps do SNMP.

WS-Management

- **WS-Management**: standard DMTF que expõe recursos geríveis através de **SOAP** sobre HTTP/HTTPS.
- Reutiliza a pilha de standards **WS-*** (WS-Addressing, WS-Transfer, WS-Eventing) já usada noutros web services empresariais.
- Implementação de referência da Microsoft: **WinRM** (Windows Remote Management), usada para gestão remota de servidores Windows.
- Vantagem sobre CIM-XML clássico: maior interoperabilidade com ferramentas e firewalls já preparadas para SOAP/HTTP(S).

SNMP vs WBEM vs WS-Management

- **SNMP**: modelo simples (MIB plana), leve, ubíquo em equipamentos de rede; fraco em modelar relações complexas.
- **WBEM/CIM**: modelo rico orientado a objetos, adequado a sistemas complexos; maior peso de implementação.
- **WS-Management**: mesma riqueza de modelo, mas transporte baseado em standards web amplamente suportados.
- Na prática, estas tecnologias coexistem: SNMP mantém-se em equipamentos de rede, WBEM/WS-Man em servidores e infraestrutura de gestão empresarial.

Gestão Fora-de-Banda: IPMI e Redfish

- Todas as tecnologias vistas até agora exigem que o sistema operativo esteja a funcionar – e é precisamente quando *não* está que a gestão remota mais faz falta.
- **Gestão fora-de-banda** (*out-of-band*): um controlador dedicado no próprio servidor, alimentado e ligado à rede independentemente do sistema operativo.
- **BMC** (Baseboard Management Controller): esse processador auxiliar, conhecido comercialmente como iDRAC (Dell), iLO (HP) ou IMM (IBM/Lenovo).
- **IPMI**: protocolo clássico para estas funções – ligar e desligar o servidor, consultar sensores, aceder à consola remota.
- **Redfish**: sucessor moderno definido pela DMTF, baseado em API REST com JSON, em substituição do IPMI.

A Tendência Atual: REST e Modelos Declarativos

- O SOAP e o XML do WS-Management deram progressivamente lugar a **APIs REST com JSON**, mais simples de consumir.
- Exemplos vistos nesta UC: a API do Proxmox (usada pelo Terraform na Aula 07) e a API do Kubernetes (usada pelo `kubect1`).
- Mudança de paradigma: em vez de executar operações de gestão uma a uma, *declara-se* o estado pretendido e o sistema reconcilia-se (Aula 05).
- **Telemetria por streaming**: em vez de o manager consultar periodicamente (polling), os dispositivos publicam métricas continuamente.
- O problema de fundo mantém-se o mesmo desde os anos 1980: gerir muitos sistemas heterogéneos de forma uniforme. Só as ferramentas mudaram.

Os Três Paradigmas de Gestão

SNMP anos 1980

Identifica por:
OID numérico

1.3.6.1.2.1.1.5.0

Resposta em tipos SMI
(Counter32, Gauge32...)

WBEM / CIM anos 1990-2000

Identifica por:
classe + chaves

CIM_OperatingSystem

Objetos com propriedades,
métodos e associações

REST hoje

Identifica por:
URL

/api2/json/nodes

Resposta em JSON;
modelo declarativo

O problema de fundo é o mesmo desde sempre: gerir muitos sistemas heterogêneos de forma uniforme.

Revisão Geral: C1 – Datacenters

- Definição NIST de cloud e as cinco características essenciais; modelos IaaS/PaaS/SaaS e de implantação (Aula 01).
- Componentes físicos, requisitos de operação (energia, arrefecimento, rede, segurança) e métricas de eficiência (PUE, DCiE).
- Tiers do Uptime Institute e níveis de redundância N, N+1, 2N (Aula 02).
- Cálculo de disponibilidade: componentes em série multiplicam-se; em paralelo, $1 - \prod(1 - A_i)$.
- Topologias three-tier e leaf-spine; tráfego norte-sul vs este-oeste.

Revisão Geral: C2 – Recursos Computacionais

- Hipervisores tipo 1 (bare-metal) vs tipo 2 (hosted); requisitos de Popek e Goldberg (Aula 03).
- Virtualização completa, paravirtualização e assistida por hardware (VT-x/AMD-V); virtio.
- Formatos de disco, thin provisioning, snapshots, clones e templates; migração ao vivo.
- Containers: namespaces isolam o que se vê; cgroups limitam o que se *consome* (Aula 04).
- Imagens em camadas (OverlayFS), arquitetura Docker (dockerd/containerd/runc) e o standard OCI.

Revisão Geral: C3 – Administração de Recursos

- Kubernetes: control plane (API server, etcd, scheduler, controller manager) e worker nodes (kubelet, kube-proxy, runtime) (Aula 05).
- Objetos centrais e o ciclo de reconciliação entre estado desejado e observado – a origem do self-healing.
- SDN: separação entre plano de controlo e plano de dados; OpenFlow e tabelas match-action (Aula 06).
- Redes overlay (VXLAN) sobre underlay físico; NFV e microsegmentação.
- IaC: idempotência, gestão de configuração (Ansible) vs provisionamento (Terraform, com state file) (Aula 07).
- Infraestrutura mutável vs imutável; CI/CD e GitOps.

Revisão Geral: C4 – Fornecimento de Serviços

- Escalabilidade vertical (scale-up) vs horizontal (scale-out); serviços stateless vs stateful (Aula 08).
- Auto-scaling: métricas, políticas reativas vs preditivas; HPA no Kubernetes.
- Balanceamento: round-robin, least connections, hashing consistente; camada 4 vs camada 7.
- Health checks (liveness vs readiness) e implantação sem downtime (rolling, blue-green, canary).
- Redundância ativa-ativa vs ativa-passiva; SPOFs; RTO e RPO (Aula 09).
- Desacoplamento por mensageria: exchanges e filas, garantias de entrega, idempotência, DLQs.

Revisão Geral: C5 – Armazenamento

- As três abstrações: bloco, ficheiro e objeto – diferentes níveis de granularidade e casos de uso (Aula 10).
- Métricas (IOPS, débito, latência); níveis de RAID e o risco do período de reconstrução.
- LVM (PV/VG/LV), thin provisioning e snapshots copy-on-write; RAID não é backup (regra 3-2-1).
- NAS (ficheiro, NFS/SMB) vs SAN (bloco, iSCSI/FC): quem gere o sistema de ficheiros em cada caso (Aula 11).
- Ceph e o algoritmo CRUSH; object storage e a API S3.
- Teorema CAP e modelo PACELC; replicação, sharding e erasure coding.

Revisão Geral: C6 – Administração de Sistemas

- SNMP: manager, agente, MIB e OIDs; a MIB-II como base normalizada (Aula 12).
- Tipos de dados (Counter vs Gauge) e o cálculo de taxas a partir de contadores.
- Versões: v1/v2c com community strings em texto simples; v3 com USM (autenticação e cifra) e VACM.
- Polling (GET) vs push (TRAP/INFORM); limiares, histerese e fadiga de alertas.
- WBEM/CIM: modelo orientado a objetos com classes, métodos e associações explícitas (esta aula).
- WS-Management sobre SOAP; gestão fora-de-banda (IPMI, Redfish); tendência atual para APIs REST.

Temas Transversais do Semestre

- **Abstração em camadas:** container sobre VM sobre hypervisor sobre hardware – cada camada esconde a de baixo e cobra algum overhead.
- **Declarar em vez de mandar fazer:** Kubernetes, Terraform e Ansible partilham o mesmo modelo de estado desejado e reconciliação.
- **Eliminar pontos únicos de falha:** redundância de energia (C1), de instâncias (C4), de discos e de nós (C5).
- **Compromissos, não soluções perfeitas:** CAP/PACELC, custo vs disponibilidade, isolamento vs desempenho.
- **Automatizar porque a escala obriga:** o fio condutor que liga SNMP nos anos 1980 ao GitOps de hoje.

Preparação para o Teste Escrito

- O teste incide sobre a totalidade dos blocos C1-C6, com peso de 40% na avaliação discreta.
- Rever, para cada bloco, os conceitos centrais e o vocabulário técnico introduzido (nomes de protocolos, camadas, algoritmos).
- Prestar atenção às ligações entre aulas destacadas ao longo do semestre (ex. SLA → disponibilidade, containers → Kubernetes → SDN).
- Rever os enunciados dos guiões práticos: várias questões de verificação exploram a mesma matéria de forma aplicada.

Obrigada.



Bom trabalho para o teste e para o miniprojeto